

Computer System

Arthur Herbette

Tuesday 17 February 2026

Contents

0.1	Memory Virtualisation	3
0.2	CPU caching & memory hierarchy	4
0.3	Process self-management	4

0.1 Memory Virtualisation

The goal of memory virtualisation is to offer a cpu to deal with more than one user on the same memory. The issue we had without and virtualisation was that. Assume I am user A and I want to steal data from user B. All I need to do is just to scan the whole memory until I find the part that belong to user B process. This is not safe. In order to solve this issue we can introduce virtualisation, the principle is to create a virtual memory for each process. Each process has its own virtual memory, which maps to a physical memory, every process therefore has its own part of the physical memory and each process doesn't know the physical memory behind.

A process references memory using **virtual** addresses, and **under the covers**, each virtual memory address is translated to a physical (real) memory address.

How Now we can see that the idea is nice however, how do we implement it? Do we allocate a contiguous part of memory for each process? Do we have some part of memory but fixed size, some part of memory that are not of fixed size?

Let us go one by one and see the pros and cons

Base and Bound In this case, the pros is that it is fairly easy to translate, all we have to do is adding the base to the virtual address. Imagine that the physical memory (the block for the process) starts at 0x80000, then when the process makes a load instruction at address 0x10, all we have to do it to look in the physical memory at address 0x80010.

The issue with that is fragmentation, imagine that we have a contiguous blablabla, for each processes P1, P2, P3, ... The issue we will have is imagine now P2 get killed, there is now a 0x400 hole in the memory. Now imagine that we want to fork a process that requests 0x900 as the size of his memory. In this case we cannot put it in there and we have to look outside. We have contiguous physical memory areas that are free but are too small to store a complete memory image. This is a result of requiring that each memory image need to be stored in a contiguous physical memory area. This results in an big inefficiency, a waste of physical memory.

Chunks Now there is a couple way of doing it in chunks we can either:

- **Segmentation:** variable sized chunks (segments)
- **Paging:** same sized chunks (frames)

The base and bound method we have seen before here is just a crude form of the segmentation, a segmentation with only one chunk.

So as we can see here those are safe and now we don't have issues as before. Let us take the example of paging, as in this case every frames has a fixed size, this implies that we when freeing memory of a process, this doesn't hurt our physical memory as every frame here can be reused for other processes, we don't have any waste of physical memory. The issue however is that they are of fixed size, this implies that imagine this size is 0x100 but we only needed 0x10 for our process then we will have 0x90 bytes that are allocated but for nothing. The same thing goes the other way, imagine that we need 0x1000000000000000 and the frames are of size 0x100 this can creates a lot of overhead on how do we allocate and access them. This is the reason on why we could have non fixed size chunk.

*Implementa-
tion*

The way we would implement this is by using a tree, I think we have already see this in the previous year c assessment but anyway. This works as a tree where each leaf is a pointer to the start of a frame. There is two variables here, the number of outgoing branch of each nodes and the depth of the tree.

The question on how do we translate from virtual to physical is as follow, let the virtual address be a 32 bits integer, we just the n MSB in order to choose the first level (so the first level of the

tree) and then either redo this or if we are at the end of the tree then use the LSB left to choose inside the chunk itself.

Where is the os? The question we might ask here is where is the OS? when I am doing a syscall I must call a function that is at a given address right? To solve this issue the OS is mapped into every process's virtual address space. When the process makes a syscall, the CPU switch to an instruction stored in some high virtual memory address.

0.2 CPU caching & memory hierarchy

As seen before a CPU instruction take around ~nsec whereas the 1 acces in the main memory take ~ 50-150nsec. In order to not slow down the cpu too much we introduce a cache inside the cpu, this cache stores the values of recently and frequently accessed memory locations. The CPU always accesses main memory through the CPU cache, what we mean by that is: when the data is not in the cache, the cache itself will access the main memory and return the value. This reduces the inefficiency resulting from CPU/main memory performance gap.

However how big should this cache be? Here we need to make some tradeof, the bigger the cache, the slower it gets. In order to have the best of both worls, we will do a hierarchy of caches with usually 3 levels, we will call those caches L1 L2 and L3 where L1 is the nearest cache to the computing core. L1 and L2 are for one core whereas L3 is for all core. The hierarchy goes as

L1 → L2 → L3 → main memory

0.3 Process self-management

As we have seen before the way how we create process is by cloning and then mutation. We `int fs = fork();` and then mutate our process in order to exec another one, e.g. `execvp("date")`.

The wait syscalls (children cleanup) The wait syscalls are used when we want to suspends the execution of the parent process while the child executes. When the child process terminates, it returns an exit status to the OS, which is then returned to the waiting parent process. Therefore it can be used as a if else statement for us to call another process and wait until it finishes before continuing.

End of a process As said in previous lecture, the way the end of a process is done is by the exit syscall: at this point, the kernel destroys the memory image, but keep the **exit status**. The process here becomes a zombie process until its parents *'reaps'* it = makes a wait syscall to **read** its **exit status**.

Adding this new zombie state to our process state diagram, the new finite state machine will look like this:

Multiple forks We can have multiple forks done at the same time, (one after another) using this we will create a tree of process. (as the child of the first work will call the next fork instruction itself etc...)

File descriptor As we have already seen that when forking, we are effectively copying all the thing image with the new thread. The question that comes up is what about the file descriptor, usually they are uniquely defined within a process. So when we will copy them, what will happen? In our case as a child process has an identical set of file descriptors, they will also be referring to the **same resources**.

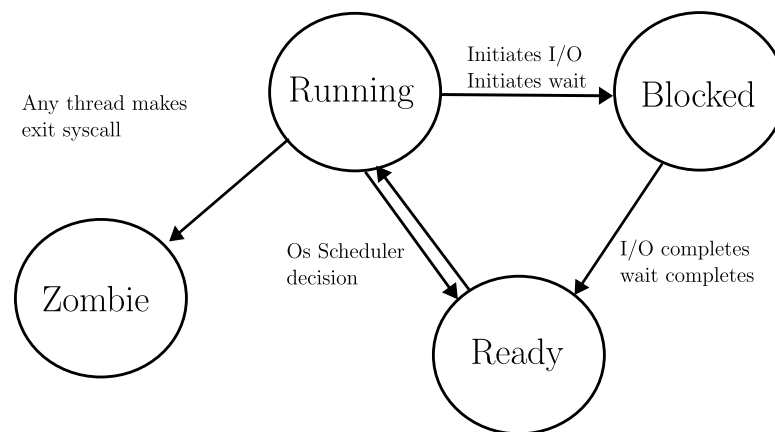


Figure 1: Process state diagram

Summary CPU caching & memory hier- archy

- Reduces the amount of time that the CPU waits for memory access to complete
- Modern architecture typically have three caches: L1, L2 and L3
- Together with the main memory, they form the computer's memory hierarchy.

Memory virtual- isation

Memory virtualisation works through address translation, implemented by the Memory Management Unit (MMU). We have three implementations possible

1. Base & Bound
2. Segmentation
3. Paging

It allows the illusion that the process thinks it is occupying the main memory alone. And it is a **safe** illusions as each process can only touch its own memory image.

0.4 Virtualization

The goal of this section is to see the Virtualization fundamentals (definition multiplexing, aggregation, emulation). We will also see an introduction to virtual machine.

0.4.1 Virtualization Fundamentals

Any problem in computer science can be solved by adding a level of indirection. But this usually creates another problem.

Virtualization applies indirection to **layering** when the layered abstraction is **compatible** with the layer it relies upon.

Remark 1. I didn't get that sentence so I will debug it using some chatgpt text.

Key terms:

- '*indirection*' = inserting an extra intermediary layer
- '*layering*' = building one abstraction on top of another
- '*compatible*' = the upper layer can behave as if the lower layer were the original expected system

The normal stack goes

Application → OS → Hardware

With Virtualization

Application → Guest OS → Virtual Machine Monitor → Hardware

The hypervisor is the 'level of indirection'

Layered & Compatible A virtualization layer exports the same (or a compatible) abstraction as the layer it relies on. It does so by hiding the physical names from the abstraction to ensure isolation.

Virtualization enforces modularity and can be applied to different types of resource (e.g., CPU, Memory...)

3 main virtualization mechanisms There exists three different way to introduce virtualization into a system.

Multiplexing We expose only a single ressource among multiple virtual entities (This is what is done with memory).

Aggregation We make multiple ressources appear as one single virtual resource (e.g., disk volumes)

Emulation We make a physical resource appear as another physical resources. This is what is being done with serial port over a USB port, or over SSH.

What you can do with virtual memory

Isolation & context switching (process abstraction)

We can enforce modularity across address spaces (as seen in previous section). To do so the OS changes the page table base (priviledge instruction). Then we flush the MMU

Demand paging

Demand paging mean, that pages are loaded into RAM only when actually needed. Suppose a program accesses a virtual page that is not currently in RAM. Then

1. MMU detects missing mapping
2. CPU raise a page fault
3. OS page fault handler runs
4. Os loads the page from disk
5. OS updates the PTE (Page Table Entry)

When we say '*change PTE and reload MMU*', what this mean in practice is that the OS updates the mapping so that the virtual page is up to date and can points to real physical page.

Copy on Write

When a `fork()` creates a child process, copying all memory immediately would be expensive. Instead:

- Parent and child initially share the same physical pages
- Both mapping are marked read-only

If one process writes:

1. Page fault occurs
2. OS allocates a new physical page
3. Os copies the old content
4. OS updates the writer's PTE

5. page becomes writable again

This is what 'copy on write' means, the actual copy happens only when modification occurs.

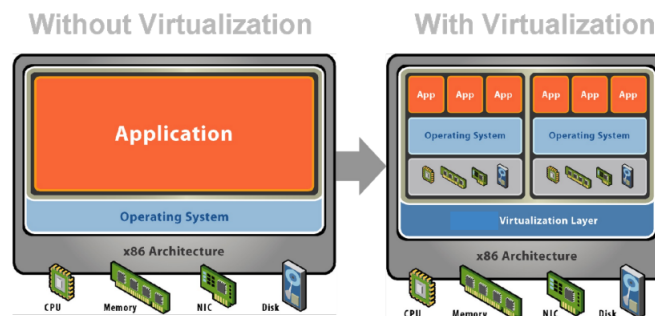
0.4.2 Aggregation and Emulation

Aggregation The primary use of aggregation is to aggregate physical resource together. Usually a computer has multiple memory module (DIMM). To be able to deal with those together, the memory controller aggregates the physical memory into a single namespace (typically contiguous).. (Here we are talking about the RAM, the stick that are plugged in the motherboard (if you are lucky to have more than one...)). But a computer also have many hard drives, we resolve that issue using RAID (Redundant Array of Inexpensive Disks) to make them appear as a single volume. To see RAID implementation and volume

Emulation The goal of virtualization via **emulation** is to use software to emulate a resource different from the underlying resource. We expose it within the OS as a virtual resource. Here are some example:

- A **RAM disk**: P = DRAM, V = disk to mount a filesystem (/tmp)
- A **Java Virtual Machine**: P=x86-64 or Arm CPU; V=Java byte code

0.4.3 Virtual Machines



The idea is to apply virtualization principle to **an entire computer system**. But first some definition

Definition 1. Virtual machine A virtual machine is an efficient, isolated duplicate of the real machine.

There is always a virtual machine monitor (or hypervisor) which is always in complete control of the system, uses the hardware directly so that programs show at most a minor decrease in speed.

Terminology

Operating system

- **Guest operating system**: running inside a virtual machine
- **Host operating system**: running directly on the hardware combined with the VMM

Memory

- Virtual memory ,as commonly understood
- **Guest physical memory**: physical memory of the VM
- **Host physical memory**: (actual) physical memory

Type 1 vs Type 2 There is two possibilities when implementing VM. The first one is having the VMM also being the host OS (done on e.g., Xen, vSphere). Or the VMM is an optional extension to the host OS (e.g. KVM and Linux)

0.4.4 VMM Construction

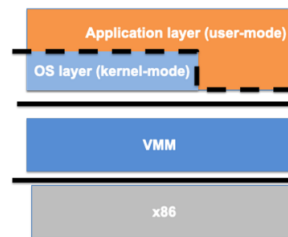
When building a VMM we will use both multiplexing and emulating. We will multiplex the CPU and Memory, (**with specific hardware support**) where

- Each VM has its own virtual CPUs (=virtual core), it requires system-level limited direct execution
- Each VM has its own guest physical memory (it relies on extended page tables)

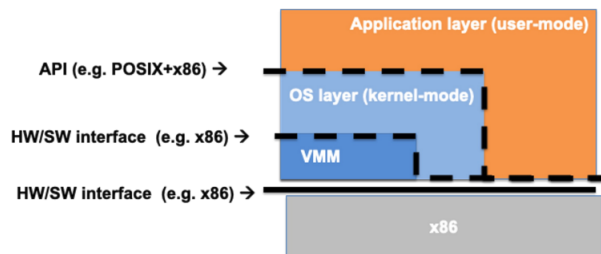
And then we will emulate all the I/O devices (virtual disk, virtual keyboard, virtual screen, virtual ethernet switch, etc.)

Limited direct execution

As seen previously, application can only execute unprivileged execution, to be able to execute privileged one, we use syscall which ask the kernel to do the jobs for us. But now this is different as we are adding a level of indirection. What we could do is to add a full layer



We will use the trick to expose the same abstraction V as the underlying resource P and therefore, everything else will still work as below. However this is not very fast right, everytime we have to go up and down the whole stack. A way to skip this is to allow the application layer and the guest OS to use the CPU directly (we will still trap some privileged instructions)



Virtual CPU –recap

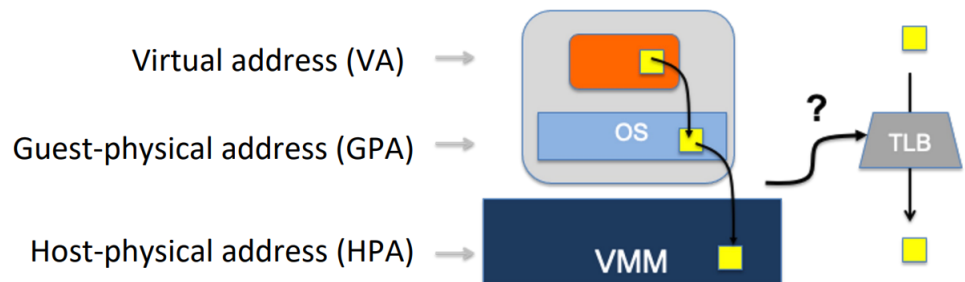
Multiplexed in space and time –very much like kernel threads

- Space: VMM schedules vCPU from different VMs on different core
- Time: VMM schedules multiple vCPUs on a limited set of physical cores

Isolation is guaranteed via:

- Non-root mode execution of the VM
- Host OS/VMM is presumed to be correct (very much like the OS)

Virtualizing physical memory



	Recall the fundamentals
	• Processor use VA • TLB stores mapping between VA and HPA • Architecture determines how TLB is loaded (e.g., page table walk)
Extended (or nested) page tables	Hardware simultaneously walks through two page table structures • One tree, controlled by the guest, containing GPA • One tree, controlled by the VMM, continuing HPA • Simultaneous walk requires quadratic number of references • Implemented in hardware, with dedicated caches to improve performance
Summary	There is two independent page table trees • It allows for independent management by the isolated entities (guest OS and VMM) • Architecture implies a quadratic number of references It also gives us a minor performance overhead • TLB hit - no performance penalty • TLB miss – minimize actual number of memory references through